Exercice – Application d'annonces sécurisée

Oct 18, 2026 · @MAHDI

1. Contexte et objectifs

Vous devez livrer **« PetitesAnnonces »**, une application web de gestion d'annonces dont toute la sécurité est centralisée sur une API Gateway. L'ensemble démarre avec une seule commande : `docker compose up`.

Le fonctionnel est volontairement minimal. L'enjeu est technique : comprendre comment un jeton émis par un Identity Provider traverse une architecture distribuée, et comment chaque couche joue son rôle.

À l'issue de l'exercice, vous saurez :

- configurer Keycloak comme Identity Provider (realm, clients, rôles, utilisateurs) ;
- implémenter le flux OAuth2 / OpenID Connect **Authorization Code + PKCE** dans une SPA Angular ;
- valider un JWT sur une API Gateway Kong (signature, émetteur, expiration), l'étendre par un plugin Lua maison et propager l'identité via des headers ;
- appliquer un **rate limiting** de 3 appels par minute ;
- reconstruire un contexte de sécurité Spring Security à partir de headers et protéger les endpoints avec `@PreAuthorize` ;
- orchestrer 5 services avec Docker Compose et isoler le réseau interne.

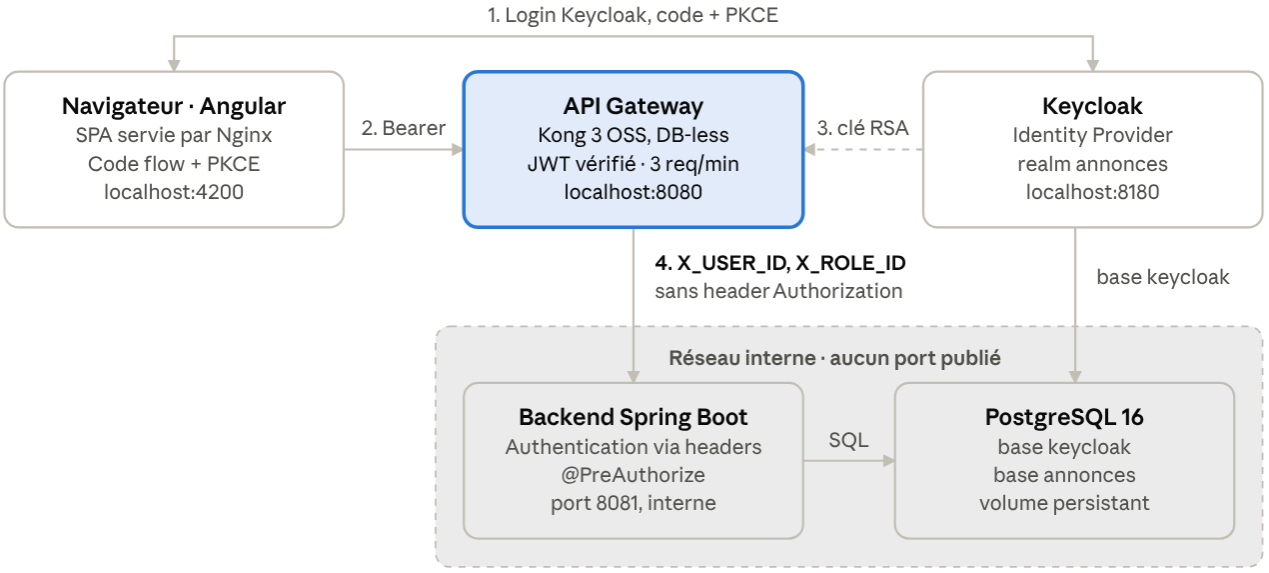
Prérequis : Java 21, Spring Boot 3, Angular 17+, notions de REST, Docker installé, notions de Lua (le plugin Kong fait une trentaine de lignes).

Durée indicative : 4 jours, seul ou en binôme.

2. Architecture cible

Cinq conteneurs, un seul point d'entrée pour l'API : la gateway. Le backend et la base sont invisibles depuis l'extérieur.

Seule la gateway lit le token ; le backend ne reçoit que deux headers



architecture cible · 5 services, 2 réseaux

Le token ne dépasse jamais la gateway : en aval, l'identité circule uniquement sous forme de headers, sur un réseau que le client ne peut pas atteindre.

Services

Service	Technologie	Port hôte	Réseaux
`frontend`	Angular 17+ servi par Nginx	4200	public
`gateway`	Kong 3.x OSS, mode DB-less (port interne 8000)	8080	public, internal
`keycloak`	Keycloak 25+	8180	public, internal
`backend`	Spring Boot 3 (Java 21), port 8081	aucun	internal
`postgres`	PostgreSQL 16	aucun	internal

Déroulé d'un appel

1. L'utilisateur ouvre l'application ; Angular le redirige vers Keycloak avec un `code_challenge` (PKCE, méthode `S256`).
2. Après le login, Keycloak renvoie un `code` ; Angular l'échange, avec le `code_verifier`, contre un access token et un refresh token.
3. Angular appelle la gateway avec `Authorization: Bearer <access_token>`.
4. Kong vérifie la signature avec la clé publique RSA du realm déclarée dans kong.yml, l'émetteur (iss) et l'expiration (exp), puis applique le rate limiting.

5. Elle supprime les headers d'identité éventuellement forgés et `Authorization`, puis ajoute `X_USER_ID` et `X_ROLE_ID`.
6. Le backend construit l'objet `Authentication`, `@PreAuthorize` autorise ou refuse, puis la requête accède à PostgreSQL.

3. Périmètre fonctionnel

Un utilisateur publie des annonces, un modérateur les valide avant leur mise en ligne, un administrateur gère les catégories. Tout endpoint exige d'être authentifié.

Rôles

Rôle	Peut faire
`USER`	Consulter les annonces publiées, créer une annonce, modifier et supprimer **ses** annonces, lister ses annonces
`MODERATEUR`	Tout ce que fait `USER` en lecture, lister les annonces en attente, publier ou rejeter une annonce, supprimer n'importe quelle annonce
`ADMIN`	Tout ce que fait `MODERATEUR`, créer et supprimer des catégories

Modèle de données

Entité	Champs	Règles
`Annonce`	`id` (UUID), `titre`, `description`, `prix`, `categorieId`, `statut`, `auteurId`, `dateCreation`, `dateModification`	Titre 5 à 100 caractères, description ⩽ 2 000 caractères, prix ⩾ 0 €, `auteurId` = valeur de `X_USER_ID`, jamais fournie par le client
`Categorie`	`id`, `libelle`	Libellé unique, 2 à 50 caractères ; 4 catégories initiales (Immobilier, Véhicules, Emploi, Multimédia)

Cycle de vie d'une annonce : `EN_ATTENTE` → `PUBLIEE` ou `REJETEE`. Toute modification par l'auteur repasse l'annonce en `EN_ATTENTE`.

Endpoints (exposés par la gateway sous `/api`)

Méthode	Chemin	Rôle requis	Comportement
GET	`/api/me`	Authentifié	Renvoie `userId` et `roles` lus dans le contexte Spring Security
GET	`/api/annonces`	`USER`	Annonces `PUBLIEE`, paginées (`page`, `size` ⩽ 50), filtre `categorieId` optionnel
GET	`/api/annonces/{id}`	`USER`	Annonce publiée, ou annonce non publiée si auteur ou `MODERATEUR` ; sinon 404
GET	`/api/annonces/mes-annonces`	`USER`	Annonces de l'utilisateur courant, tous statuts
POST	`/api/annonces`	`USER`	Crée l'annonce en `EN_ATTENTE`, renvoie 201 + `Location`
PUT	`/api/annonces/{id}`	`USER` + auteur	Modifie l'annonce, statut remis à `EN_ATTENTE`
DELETE	`/api/annonces/{id}`	Auteur, `MODERATEUR` ou `ADMIN`	Supprime, renvoie 204
GET	`/api/moderation/annonces`	`MODERATEUR`	Annonces `EN_ATTENTE`, plus anciennes d'abord

Méthode	Chemin	Rôle requis	Comportement
PATCH	`/api/moderation/annonces/{id}/publier`	`MODERATEUR`	Passe en `PUBLIEE`
PATCH	`/api/moderation/annonces/{id}/rejeter`	`MODERATEUR`	Passe en `REJETEE`, corps `{ "motif": "…" }` obligatoire
GET	`/api/categories`	`USER`	Liste des catégories
POST	`/api/categories`	`ADMIN`	Crée une catégorie
DELETE	`/api/categories/{id}`	`ADMIN`	Supprime une catégorie sans annonce, sinon 409

La hiérarchie des rôles est portée par Keycloak : un `MODERATEUR` possède aussi `USER`, un `ADMIN` possède aussi `MODERATEUR` et `USER` (rôles composites).

4. Travail demandé par composant

Chaque composant a une responsabilité unique : Keycloak authentifie, la gateway contrôle, le backend autorise.

4.1 Docker Compose

- Un fichier `docker-compose.yml` à la racine démarre les 5 services : `postgres`, `keycloak`, `gateway` (Kong), `backend`, `frontend`.
- Deux réseaux : `public` (frontend, gateway, keycloak) et `internal` (gateway, backend, postgres, keycloak).
- Ports publiés sur l'hôte : `4200` (frontend), `8080` (proxy Kong, port interne 8000), `8180` (keycloak). **Le backend, PostgreSQL et l'Admin API de Kong ne publient aucun port.**
- Chaque service a un `healthcheck` (pour Kong : `kong health`) ; les `depends_on` utilisent `condition: service_healthy`.
- Mots de passe et secrets dans un fichier `.env` (fourni en `.env.example`, `.env` ignoré par Git).
- Images applicatives construites par des Dockerfile multi-stage (build Maven / npm, puis image d'exécution légère) ; Kong utilise l'image officielle, sans build.

4.2 PostgreSQL

- Une instance PostgreSQL 16 avec deux bases : `keycloak` et `annonces`, créées par un script d'initialisation.
- Données persistées dans un volume nommé.
- Le schéma `annonces` est géré par Flyway (migrations versionnées + catégories initiales).

4.3 Keycloak (Identity Provider)

- Keycloak 25+ en mode `start-dev`, base PostgreSQL, realm importé au démarrage depuis `keycloak/realm-annonces.json`.
- Realm `annonces` avec les rôles de realm `USER`, `MODERATEUR`, `ADMIN` (composites, voir §3).
- Client `annonces-front` : client **public**, *Standard flow* seul activé (implicit et direct access grants désactivés), PKCE `S256` obligatoire, redirect URI `http://localhost:4200/*`, web origin `http://localhost:4200`.
- Durée de vie de l'access token : **5 minutes** (pour pouvoir tester l'expiration).
- Utilisateurs de test : `alice` et `bob` (`USER`), `modo` (`MODERATEUR`), `admin` (`ADMIN`).

4.4 Frontend Angular

- Angular 17+, servi en production par Nginx dans son conteneur.
- Authentification par **Authorization Code + PKCE** avec une librairie OIDC (`angular-oauth2-oidc` ou `keycloak-angular`).
- Tokens conservés en mémoire ou en `sessionStorage`, jamais en `localStorage`.
- Un `HttpInterceptor` ajoute `Authorization: Bearer <access_token>` aux seuls appels vers la gateway.
- Renouvellement du token avant expiration via le refresh token.
- Déconnexion via l'endpoint `end_session` de Keycloak.
- Écrans : liste des annonces, détail, mes annonces, formulaire création / modification, modération (`MODERATEUR`), catégories (`ADMIN`).
- Les menus et routes sont masqués selon les rôles du token (guards). Ce filtrage est un confort d'affichage, pas une sécurité.
- Gestion des erreurs : 401 → retour au login, 403 → message « accès refusé », 429 → message indiquant le délai d'attente lu dans `Retry-After`.

4.5 API Gateway (Kong OSS en mode DB-less)

La gateway est le **seul point d'entrée** de l'API et le seul composant qui lit le JWT. On utilise l'image officielle `kong:3.x` (édition open source), sans base de données : toute la configuration tient dans un fichier déclaratif `kong/kong.yml` versionné, chargé via `KONG_DECLARATIVE_CONFIG`.

1. Déclarer un *service* `annonces-backend` (`http://backend:8081`) et une *route* `/api` qui pointe dessus.
2. Activer le plugin `cors` : origine `http://localhost:4200` uniquement, méthodes GET, POST, PUT, PATCH, DELETE, header `Authorization` autorisé, `Retry-After` exposé. Les preflight `OPTIONS` sont traités par ce plugin, avant toute authentification et tout décompte.
3. Activer le plugin `jwt` sur la route pour valider le token reçu dans `Authorization` :
   - `key_claim_name: iss` : le token doit provenir du realm `annonces` ;
   - `claims_to_verify: [exp]` : le token ne doit pas être expiré ;
   - un *consumer* `keycloak-annonces` porteur d'un credential `jwt` dont la `key` est l'URL du realm (`http://localhost:8180/realms/annonces`), l'algorithme `RS256` et la `rsa_public_key` la clé publique du realm au format PEM ;
   - token absent, mal signé, expiré ou d'un autre émetteur : 401.
4. Écrire un **plugin Kong maison en Lua**, `identity-headers`, chargé via `KONG_PLUGINS=bundled,identity-headers` et monté par volume. Sa priorité doit le placer **après** `jwt` (1450) et **avant** `rate-limiting` (910). Il :
   - supprime tout header `X_USER_ID` ou `X_ROLE_ID` envoyé par le client (anti-usurpation) ;
   - lit le token déjà validé par `jwt` et ajoute `X_USER_ID` = claim `sub` ;
   - ajoute `X_ROLE_ID` = rôles du claim `realm_access.roles` limités à `USER`, `MODERATEUR`, `ADMIN`, séparés par des virgules (ex. `MODERATEUR,USER`) ;
   - retire le header `Authorization` : le backend ne voit jamais le token.
5. Activer le plugin `rate-limiting` : `minute: 3`, `policy: local`, `limit_by: header`, `header_name: X_USER_ID`. Le compteur est donc par utilisateur, sur tous les endpoints `/api/**`. Au-delà : 429 avec header `Retry-After`.
6. Ne pas publier l'Admin API de Kong (port 8001) sur l'hôte.

4.6 Backend Java (Spring Boot 3)

- Java 21, Spring Boot 3, Spring Web, Spring Security, Spring Data JPA, Bean Validation, Flyway.

- **Aucune dépendance OAuth2 / JWT / Keycloak** : le backend fait confiance à la gateway et lit uniquement les headers.
- Un filtre `OncePerRequestFilter` lit `X_USER_ID` et `X_ROLE_ID`, construit un objet `Authentication` (principal = userId, autorités = `ROLE_` + chaque rôle) et le place dans le `SecurityContextHolder`.
- Headers absents ou vides → 401.
- Configuration *stateless* : pas de session, pas de formulaire de login, CSRF désactivé.
- Sécurité déclarative avec `@EnableMethodSecurity` et `@PreAuthorize` sur les méthodes de service ou de contrôleur (ex. `hasRole('MODERATEUR')`).
- Contrôle de propriété par expression, ex. `@PreAuthorize("@annonceSecurity.estAuteur(#id, authentication)")`.
- Erreurs au format JSON `ProblemDetail` (RFC 9457) : 400, 401, 403, 404, 409.
- Tests d'intégration avec `@SpringBootTest` + Testcontainers PostgreSQL, en injectant directement les headers.

> **Pièges classiques à anticiper**
>
> - Le plugin `jwt` de Kong OSS ne lit pas le JWKS : la clé publique est figée dans `kong.yml`. Le realm exporté doit donc embarquer une paire de clés RSA fixe, sinon Keycloak en génère une nouvelle et tous les tokens sont rejetés.
> - L'`iss` du token vaut l'URL vue par le navigateur (`localhost:8180`), pas `keycloak:8080` : la `key` du credential doit être exactement cette valeur (fixer `KC_HOSTNAME`).
> - Le plugin `openid-connect` de Kong est réservé à l'édition Enterprise : il est interdit ici.
> - Nginx, sur lequel repose Kong, ignore par défaut les headers client contenant un `_` : c'est une protection de plus, mais le plugin doit tout de même les supprimer explicitement.

5. Critères d'acceptation

L'exercice est validé quand les 48 critères ci-dessous passent. Chacun doit être démontrable en soutenance (navigateur, `curl` ou test automatisé).

5.1 Déploiement

ID	Étant donné	Quand	Alors
DEP-01	Un poste avec Docker et le dépôt cloné, `.env` copié depuis `.env.example`	On lance `docker compose up -d --build`	Les 5 conteneurs sont `healthy` en moins de 3 minutes, sans action manuelle
DEP-02	La stack démarrée	On appelle `http://localhost:8081` ou le port 5432 depuis l'hôte	La connexion échoue : backend et base ne sont pas exposés
DEP-03	La stack démarrée	On ouvre `http://localhost:8180`	La console Keycloak affiche le realm `annonces` importé, avec clients, rôles et utilisateurs
DEP-04	Des annonces créées	On lance `docker compose down` puis `up`	Les annonces et la configuration Keycloak sont conservées
DEP-05	Le dépôt	On cherche un mot de passe en clair	Aucun secret n'est commité hors `.env.example` (valeurs factices)

5.2 Keycloak et flux Authorization Code + PKCE

ID	Étant donné	Quand	Alors
KC-01	Un utilisateur non connecté	Il ouvre `http://localhost:4200`	Il est redirigé vers la page de login Keycloak du realm `annonces`
KC-02	La redirection vers Keycloak	On inspecte l'URL d'autorisation	Elle contient `response_type=code`, `code_challenge` et `code_challenge_method=S256`
KC-03	Le retour sur l'application	On inspecte le réseau	Le front échange le `code` contre les tokens sur l'endpoint `/token` avec le `code_verifier` ; aucun token n'apparaît dans l'URL

ID	Étant donné	Quand	Alors
KC-04	Le client `annonces-front`	On tente un appel `grant_type=password` ou un flux implicit	Keycloak refuse : seul le Standard flow est autorisé
KC-05	`modo` connecté	On décode son access token	`realm_access.roles` contient `MODERATEUR` et `USER`
KC-06	Un utilisateur connecté	Il clique sur « Se déconnecter »	La session Keycloak est fermée ; un retour sur l'application redemande le login

5.3 Frontend

ID	Étant donné	Quand	Alors
FR-01	`alice` connectée	Elle consulte les annonces	Chaque appel à la gateway porte `Authorization: Bearer …` ; aucun appel à Keycloak ne le porte inutilement
FR-02	`alice` connectée	On inspecte le `localStorage`	Aucun token n'y est stocké
FR-03	`alice` (`USER`) connectée	Elle affiche le menu	Les entrées « Modération » et « Catégories » sont absentes ; l'URL `/moderation` saisie à la main la redirige
FR-04	`alice` connectée depuis plus de 5 minutes	Elle continue de naviguer	Le token est renouvelé sans retour à la page de login
FR-05	Une réponse 429 de la gateway	L'écran l'affiche	Un message indique le nombre de secondes à attendre, lu dans `Retry-After`
FR-06	Une réponse 403	L'écran l'affiche	Un message « accès refusé » s'affiche, sans déconnexion

5.4 Gateway : authentification centralisée

ID	Étant donné	Quand	Alors
GW-01	Aucun header `Authorization`	On appelle `GET /api/annonces`	401, la requête n'atteint pas le backend (vérifiable dans ses logs)
GW-02	Un token dont la signature a été altérée	On appelle un endpoint	401
GW-03	Un token expiré (plus de 5 minutes)	On appelle un endpoint	401
GW-04	Un token valide émis par un autre realm ou un autre Keycloak	On appelle un endpoint	401 : l'émetteur `iss` ne correspond à aucun credential Kong
GW-05	Un token valide d'`alice`	On appelle `GET /api/me`	200, `userId` = `sub` d'`alice`, `roles` = `["USER"]`
GW-06	Un token valide d'`alice` et un header `X_USER_ID` forgé avec le `sub` de `bob`	On appelle `GET /api/me`	La réponse contient le `sub` d'`alice` : le header forgé a été supprimé
GW-07	Un token valide d'`alice` et `X_ROLE_ID: ADMIN` forgé	On appelle `POST /api/categories`	403
GW-08	Une requête valide	Le backend la reçoit (log de debug)	Elle contient `X_USER_ID` et `X_ROLE_ID` mais pas `Authorization`
GW-09	Un appel depuis l'origine `http://evil.local`	Le navigateur envoie le preflight	Kong ne renvoie pas `Access-Control-Allow-Origin` pour cette origine
GW-10	La stack démarrée	On appelle `http://localhost:8001` depuis l'hôte	La connexion échoue : l'Admin API de Kong n'est pas exposée

ID	Étant donné	Quand	Alors
GW-11	Le dépôt	On inspecte la configuration Kong	Elle tient entièrement dans `kong/kong.yml` (DB-less), le plugin `identity-headers` est dans `kong/plugins/` et seuls des plugins de l'édition open source sont utilisés

5.5 Gateway : rate limiting

ID	Étant donné	Quand	Alors
RL-01	`alice` authentifiée	Elle fait 3 appels en moins d'une minute	Les 3 appels répondent normalement
RL-02	`alice` a déjà fait 3 appels dans la minute	Elle fait un 4e appel, sur n'importe quel endpoint `/api/**`	429 avec un header `Retry-After` en secondes ; le backend n'est pas appelé
RL-03	`alice` a reçu un 429	Elle attend le délai indiqué puis rappelle	L'appel répond normalement
RL-04	`alice` est bloquée	`bob` fait un appel	`bob` n'est pas bloqué : le compteur est par utilisateur
RL-05	`alice` est bloquée	Le navigateur envoie un preflight `OPTIONS`	Le preflight répond sans être décompté

5.6 Backend : contexte Spring Security

ID	Étant donné	Quand	Alors
BK-01	Le code du backend	On inspecte le `pom.xml`	Aucune dépendance `spring-boot-starter-oauth2-*`, `jjwt`, `nimbus` ou Keycloak
BK-02	Une requête sans `X_USER_ID` (test d'intégration)	Elle atteint un contrôleur	401

ID	Étant donné	Quand	Alors
BK-03	`X_USER_ID: u1` et `X_ROLE_ID: MODERATEUR,USER`	On lit `SecurityContextHolder` dans un contrôleur	Principal = `u1`, autorités = `ROLE_MODERATEUR`, `ROLE_USER`
BK-04	Les endpoints du §3	On lit le code	Chaque règle d'accès est exprimée par `@PreAuthorize`, pas par des `if` sur les rôles dans le code métier
BK-05	Deux requêtes successives d'utilisateurs différents	On compare leurs contextes	Aucune fuite : le contexte est vidé en fin de requête, aucune session n'est créée

5.7 Règles métier et autorisations

ID	Étant donné	Quand	Alors
MT-01	`alice` connectée	Elle crée une annonce valide	201, statut `EN_ATTENTE`, `auteurId` = son `sub` même si le corps envoie un autre `auteurId`
MT-02	Une annonce `EN_ATTENTE` d'`alice`	`bob` appelle `GET /api/annonces/{id}`	404
MT-03	Une annonce d'`alice`	`bob` tente `PUT` ou `DELETE`	403
MT-04	Une annonce `PUBLIEE` d'`alice`	`alice` la modifie	200, statut revenu à `EN_ATTENTE`, l'annonce disparaît de la liste publique
MT-05	`alice` (`USER`)	Elle appelle `PATCH /api/moderation/annonces/{id}/publier`	403
MT-06	`modo` connecté	Il publie une annonce en attente	200, l'annonce apparaît dans `GET /api/annonces`

ID	Étant donné	Quand	Alors
MT-07	`modo` connecté	Il rejette sans `motif`	400 au format `ProblemDetail`
MT-08	`modo` (`MODERATEUR`)	Il appelle `POST /api/categories`	403 ; le même appel par `admin` renvoie 201
MT-09	Une catégorie utilisée par une annonce	`admin` la supprime	409
MT-10	Un titre de 3 caractères ou un prix négatif	On crée l'annonce	400 détaillant le champ en erreur

6. Livrables et évaluation

Livrables

- Un dépôt Git avec l'arborescence `frontend/`, `backend/`, `kong/kong.yml`, `kong/plugins/identity-headers/` (`handler.lua`, `schema.lua`), `keycloak/realm-annonces.json`, `postgres/init.sql`, `docker-compose.yml`, `.env.example`.
- Un `README.md` : démarrage en une commande, comptes de test, schéma d'architecture, choix techniques justifiés, procédure d'extraction de la clé publique du realm.
- Une collection Bruno ou Postman (ou un script `curl`) couvrant les critères GW, RL et MT.
- Les tests automatisés du backend (critères BK et MT) et un script de test de la gateway (token absent, expiré, forgé, rate limit).
- Une soutenance de 20 minutes : démonstration en direct, puis questions.

Contraintes

- Commits réguliers, messages explicites ; pas de commit unique final.
- Aucune règle d'autorisation dans le front ne remplace une règle côté serveur.
- Le backend ne doit jamais appeler Keycloak.

Bonus

- Rate limiting partagé entre 2 instances de Kong, avec `policy: redis` et un conteneur Redis.
- Vérification du claim `azp` (client `annonces-front`) dans le plugin `identity-headers`, en plus de `iss` et `exp`.
- Secret partagé ou mTLS entre Kong et le backend, pour que le backend refuse un appel ne venant pas de la gateway.
- Récupération automatique de la clé publique au démarrage (script d'init qui lit le JWKS et génère `kong.yml`).
- Traçabilité : plugin `correlation-id` de Kong, identifiant repris dans les logs du backend.

Grille d'évaluation

Domaine	Critères	Points
Déploiement Docker Compose	DEP-01 à DEP-05	15
Keycloak et flux PKCE	KC-01 à KC-06	15
Frontend Angular	FR-01 à FR-06	15
Gateway : authentification et headers	GW-01 à GW-11	20
Gateway : rate limiting	RL-01 à RL-05	10
Backend : contexte Spring Security	BK-01 à BK-05	10
Règles métier	MT-01 à MT-10	10
Qualité : tests, README, soutenance	—	5
Total		**100**

Seuil de validation : 70 points, avec GW-01 à GW-07 et BK-01 à BK-03 obligatoirement validés. Les bonus rapportent jusqu'à 10 points supplémentaires.